STMicroelectronics

Unsupervised learning methods for clustering microelectronic components.

Bastien Roure

Supervised by : Philippe Rochette Anthony Aoun

Abstract—The aim of this work is to provide an unsupervised algorithm for clustering microelectronic components for the company STMicroelectronics. For a company of this scale, we are talking about hundreds of thousands of items of data to be sorted. The objective is therefore to easily carry out competitor studies, search for equivalents in the market or within the company itself, or identify similar elements to observe obsolescence. To achieve this result, we will study different clustering algorithms such as **K-Means**/**K-Prototypes** or **Spectral clustering**, then evaluate different metrics like **Minkowski** or **Manhattan**, in order to compare them with the **Euclidean distance** and its derivatives. Finally, we'll look at some algorithms for visualizing data, such as **PCA** and **t-SNE**. This paper is therefore an overview of the different methods and metrics that were studied during this short one-month internship. Some results are also presented to highlight the beginnings of the results obtained.

1 Introduction

In large companies such as STMicroelectronics, managing and analyzing vast amounts of product data, both internal and external, poses considerable challenges. This data can include thousands of items, components developed in-house, as well as those offered by competitors. Manually identifying similarities, differences, or potential overlaps between products can quickly become not only a waste of time, but also a source of costly errors. Yet it is essential to maintain a complete and up-to-date understanding of the market as a whole. For example, knowing whether a competitor offers a similar product at a lower price can influence strategic decisions such as pricing or even product development. Similarly, when a customer asks whether the company supplies a component equivalent to that of a competitor, it is imperative to be able to respond with confidence, speed, and accuracy.

To address these challenges, the use of automated data analysis tools becomes indispensable. In this context, clustering algorithms provide a powerful solution. These unsupervised learning techniques allow us to explore and structure our data by automatically grouping similar items based on their characteristics. The resulting groups, called 'clusters', offer a way to interpret patterns and relationships within the data without the need for predefined labels or categories.

The main objective of this project is to apply clustering techniques to a heterogeneous dataset containing products from STMicroelectronics and various competitors. One of the key difficulties lies in the fact that we have no prior information about which product belongs to which group. The challenge, therefore, is to define meaningful similarities between products and to ensure a clear separation between clusters, so that each group reflects a coherent set of comparable items. Through this approach, we aim to enhance both market analysis and internal decision-making processes by offering a structured, data-driven perspective on product equivalence and competition.

2 Background

2.1 Clustering algorithms

There are numerous unsupervised clustering algorithms, such as K-means, Gaussian Mixture Models, Mean of Deviations, DBSCAN, Hierarchical Clustering, Spectral Clustering, BIRCH, etc. These different clustering algorithms adopt a variety of approaches : some are categorical, such as K-Means or BIRCH, which explicitly partition the data into groups. Others are probabilistic, such as Gaussian Mixture Models (GMM), which estimate the probability of membership of each cluster. Still others are based on density, such as DBSCAN, or topological structure, such as Hierarchical Clustering or Spectral Clustering, which exploit similarity graphs or inter-object distances respectively. Finally, more heuristic methods such as Average Deviation are based on simpler statistical criteria to assess the cohesion or separation of clusters.

We first turned our attention to one of the most widely used clustering algorithms: **K-Means**.

2.1.1 K-Means

Let $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^m$ be a set of n data points. Given a positive integer $k < n$, the goal of the **K-Means** algorithm is to partition X into k disjoint clusters $C = \{C_1, C_2, \ldots, C_k\}$ such that the following objective function is minimized :

$$\arg\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} d(x - \mu_i),$$

where $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ is the centroid (mean) of cluster C_i.

By default $d = \| \cdot \|_2$, the Euclidean distance, which is perhaps not the most relevant metric. With this approach, we assume that the space generated by our data is a Euclidean metric space. However, if the data are not naturally represented in such a space (for example, if they contain categorical variables or dimensions of a heterogeneous nature), the Euclidean distance may not be adequate. The challenge is therefore to find a suitable metric, such as $d : \mathbb{R}^m \times \mathbb{R}^m \mapsto \mathbb{R}^+$ which is relevant to the space generated by our data. We will study in sec 2.2 different metrics to define the one best suited to our data.

Here are the different steps involved in this algorithm:

Algorithm 1 K-Means

Require: Data points $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^m$, number of clusters k

Ensure: Cluster assignments $C_1, \ldots, C_k$, centroids $\mu_1, \ldots, \mu_k$

1: Initialize centroids $\mu_1, \ldots, \mu_k$ (e.g., randomly select k points from X)
2: **repeat**
3: **for** each point $x_j \in X$ **do**
4: Assign x_j to the cluster C_i whose centroid μ_i is closest:

$$C_i \leftarrow C_i \cup \{x_j\} \quad \text{where } i = \arg\min_{l} d(x_j - \mu_l)$$

5: **end for**
6: **for** each cluster C_i **do**
7: Update centroid μ_i as the mean of its assigned points:

$$\mu_i \leftarrow \frac{1}{|C_i|} \sum_{x \in C_i} x$$

8: **end for**
9: **until** Cluster assignments no longer change

In the case of this internship, the data consisted of 34 features, 6 of which were categorical. We therefore used a variant of K-Means called **K-Prototypes** proposed by Zhexue Huang in 1998 [1]. This allows us to process both numerical and categorical data.
The main difference between these two algorithms is that here we are not calculating a distance for categorical features, but a dissimilarity. This simple dissimilarity function is defined as follows :

$$\delta(x,y) = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{else} \end{cases}$$

The total distance is therefore :

$$d_{tot}(x,y) = \sum_{i \in num} d(x_i, y_i) + \sum_{j \in cat} \beta \cdot \delta(x_j, y_j)$$

Here, $x, y \in \mathbb{R}^m$, with i the numerical features and j the categorical. β is the balancing parameter to weight the importance of categorical attributes against numerical ones.

The important choice for this algorithm is the hyperparameter K, the number of clusters. There are several ways for finding this value. One of the most commonly heuristic method used is called the 'elbow method'. The idea behind this algorithm is to choose the good value of K by looking at how the sum of intra-cluster distances (Within-Cluster Sum of Squares) evolves.

$$\text{WCSS} = \sum_{i=1}^{K} \sum_{x \in C_i} d(x, \mu_i)$$

With K the number of clusters, C_i the i^{th} cluster, x the data points in the cluster C_i and μ_i the centroid of the cluster i. The metric d here is the same as the one used in K-Means.

As **K** increases, **WCSS** always decreases (because more centroids are added, so the points are closer to their center). But there comes a point when the gain becomes marginal: adding one more cluster hardly brings any improvement. The **" elbow "** in the curve (change in slope; figure1) generally corresponds to the optimum value of **K**.

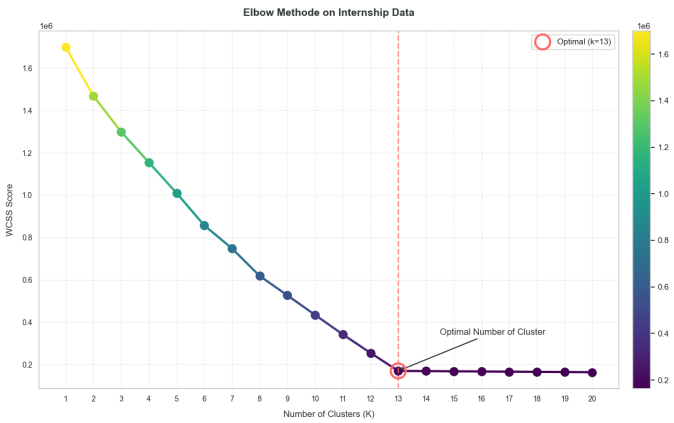

Fig. 1. Elbow method on the 500K dataset from STmicroelectronics. This plot shows an optimal cluster value of 13.

2.1.2 Spectral Clustering

Another approach studied during this internship was **'spectral' clustering**. The general idea behind it is to build an undirected weighted graph and to map the points (the graph's vertices) into the spectral space, spanned by the eigenvectors of the Laplacian matrix.

The aim is to find a partition of the graph, built from the data, that minimizes the cuts (fig 2) between clusters, while keeping them balanced. This structure is captured by the spectrum (eigenvalues and eigenvectors) of the Laplacian matrix, derived from the Similarity matrix.

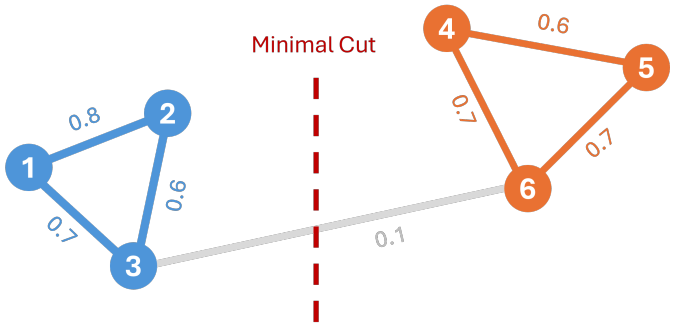

Fig. 2. Example of a weighted graph and a minimal cut representing the separation of two clusters.

Let's delve into the mathematics behind this algorithm:

Once again, let's assume : $X = (x_1, x_2, ..., x_n) \subset \mathbb{R}^m$, we want to build a **weighted graph** representing the similarities between our data. As a result, we have a similarity matrix, denoted $S \in \mathbb{R}^{n \times n}$. In the literature, one way of obtaining a similarity measure between two objects is via the **Gaussian Kernel**.

$$S_{i,j} = \exp\left(\frac{-d^2(x_i, x_j)}{2\sigma^2}\right)$$

From S, we can construct the degree matrix D which gives us information on the importance of the connections a point possesses.

$$D_{i,i} = \sum_j S_{i,j}$$

We can then calculate the Laplacian of a graph using the formula: $L = D - S$. This measures, for each node, the extent to which its value differs from that of its neighbors by subtracting from the total sum of its connections (D) the direct effect of its interactions with them (S); it is therefore a measure of local variation. Other formulas exist to approximate the Laplacian (table 1), and can be used to model various properties of the graph, depending on variation requirements.

TABLE 1
Types of Graph Laplacians

Type	Formula
Unnormalized	$L = D - W$
Symmetric normalized	$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$
Random walk normalized	$L_{\text{rw}} = D^{-1} L = I - D^{-1} W$

Next we need to calculate the k smallest (non-zero) eigenvectors of L, which will give us a matrix $U \in \mathbb{R}^{n \times k}$, where each row $u_i \in \mathbb{R}^k$ is a new representation of the point x_i in a spectral space. The final step is to apply **K-Means** to the U lines. This allows us to group together points that are close in spectral space, i.e. well connected in the initial graph.

note : This method could not be tested during this internship, but looks promising on the complex data provided.

2.2 Metrics

In many cases, the data reside in a manifold or subspace immersed in a high-dimensional space, where the intrinsic geometry is not Euclidean. It is therefore rigorous to consider other metrics that can better capture the real structure of the data space. Her is an overview of the different metrics that caught our attention during this internship.

The 3 main numerical ones are :

- The **Euclidean Distance** :

$$\|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

  This distance is a must, the natural measure between two points in a Euclidean space.

- The **Manhattan Distance** :

$$\|x - y\|_1 = \sum_{i=1}^{n} |x_i - y_i|$$

  This distance, also known as the L1 norm, can be seen as a distance along the lines of a grid.

- The **Chebyshev Distance** :

$$\|x - y\|_\infty = \max_{i=1}^{n} |x_i - y_i|$$

  The maximal distance on a dimension

- (Bonus) The **Minkowski Distance** :

$$\|x - y\|_p = (\sum_{i=1}^{n} |x_i - y_i|^p)^{\frac{1}{p}}$$

  Generalization of previous distances to vary sensitivity to large differences between components (figure 3).

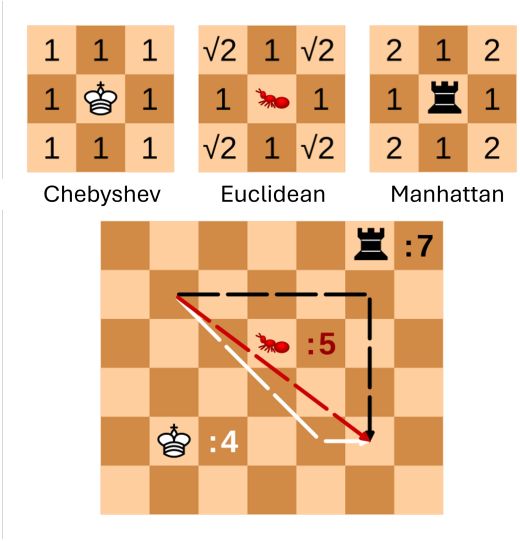

Fig. 3. Comparison of different distances presented. Source: `https://en.wikipedia.org/wiki/Minkowski_distance`

There are a multitude of distances that could not be studied at length. These include Pearson, Cosine, Canberra, Mahalanobis, etc. Enough to make another internship.

As we said earlier, in our data there are features that are not numerical but categorical, so we also had to look at the metrics that would enable us to calculate a distance for them.

The main ones that caught our attention were :

- The **Jaccard Distance** :

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

  This distance compares two sets and measures the difference between what they have in common and what they have in total. Widely used to compare binary sets or words.

- The **Dice Distance** :

$$d(A, B) = 1 - \frac{2|A \cap B|}{|A| + |B|}$$

  Similar to Jaccard, but gives greater importance to the intersection.

- The **Hamming Distance** :

$$d(x, y) = \sum_{i=1}^{n} \mathbf{1}_{x_i \neq y_i}$$

  Compare two binary strings (in our case words) of the same length and count the number of positions where the bits are different.

Here too, numerous distances exist, including Kulczynski1, Rogers-Tanimoto, Russell-Rao, Sokal-Michener, Sokal-Sneath and Yule. In our tests, the various categorical metrics did not produce any significant differences.

3 Data

During this internship, the given data looked like this: we had a dataset of around 500k data points of 34 features, some of which were encrypted for security reasons. Data living in a space with more than 3 dimensions is difficult to visualize. So, there are various algorithms for distributing them in 2D or 3D space.

3.1 Principal Component Analysis

The Principal Component Analysis (PCA) [2] algorithm plays a pivotal role in reducing the dimensionality of complex datasets by simplifying their interpretation. Principal Component Analysis is a statistical method that transforms high-dimensional data into a lower-dimensional form while preserving the most important information. It accomplishes this by identifying new axes, called principal components, along which the data vary the most. These components are orthogonal to each other, meaning they are uncorrelated, making them a powerful tool for dimensionality reduction. This reduction inevitably leads to a loss of information. It can therefore be used, but the result is very limited.

The Mathematics Behind PCA :

Always with the same dataset $X = (x_1, x_2, ..., x_n) \subset \mathbb{R}^m$, the first step is to center the data:

$$\forall i \in n, \bar{x}_i = x_i - \frac{1}{n}\sum_{j \in n} x_j$$

Let $\bar{X} = (\bar{x}_1, \bar{x}_2, ..., \bar{x}_n)$ be the centered dataset.

Let's now compute the covariance matrix. Covariance is used to understand linear relationships between variables. In other words, it measures how two variables vary from one another. $\Sigma \in \mathbb{R}^{m \times m}$, the covariance matrix of $\bar{X}$, is defined by :

$$\Sigma = \frac{1}{n-1}\bar{X}^T \cdot \bar{X}$$

The next step is to compute the eigenvalues and eigenvectors of the covariance matrix. The Eigen Vectors are the direction where the variance of the data is maximized. Each Eigen Vector has an Eigen Value. The higher this value, the greater the variance along this vector.

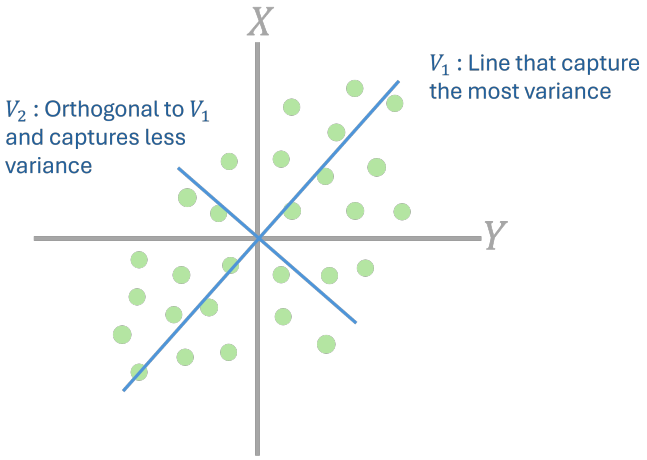

Fig. 4. Visualization of the 2 greatest Eigen Vectors

Let $w \in \mathbb{R}^m$, a unit vector (Eigen Vector). The objective is to maximize the variance of the data projected on w : $Var(\bar{X}w) = w^T \cdot \Sigma \cdot w$

Let $V \in \mathbb{R}^{m \times m}$ and $\Lambda \in \mathbb{R}^{m \times m}$, be respectively, the matrix of Eigen Vectors and the matrix of Eigen Values (where $\Lambda = diag(\lambda_1, \lambda_2, ..., \lambda_n)$, where $\lambda_1 \geq \lambda_2 \geq ... \geq \lambda_m$)

Finally, project data on $k < d$ dimensions

$$\boxed{X_{proj} = \bar{X} \cdot V_k}$$

where V_k is a matrix of the k first eigen vectors (figure 4).

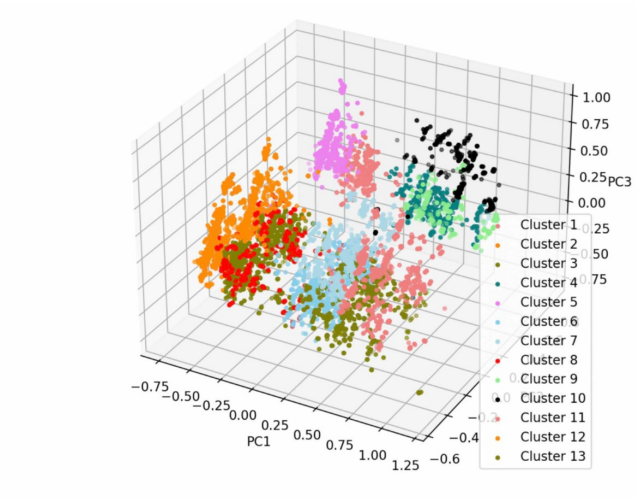

Fig. 5. PCA plot on the data given for the internship.

When used on our STMicroelectronics data (figure 5), the PCA algorithm did not reveal any relevant information about the distribution of the data.

3.2 T-Distributed Stochastic Neighbor Embedding

Another algorithm for visualizing data and improving dimensionality reduction techniques compared to PCA is called t-SNE. [3]. PCA works better on linear data while t-SNE has no such restriction. Whether the data are linear or non-linear, t-SNE performs well.

What is the difference with PCA ? The idea here is not to look at the eigenvectors to give us an indication of the greatest variance of the data, but directly to look at the proximity of two points in high-dimensional space and represent their similarity as a probability. Then, it constructs a similar probability distribution in the lower-dimensional space and minimizes the difference between the two distributions using gradient descent. This effectively captures the local structure of the data, which can be useful for visualizing complex datasets, and potentially discovering significant patterns.

To remember: if two similar points are close in the high-dimensional space, we want to preserve this local relationship after the dimensionality has been reduced. Similarly, if two data points are different or distant from each other in the original space, they must remain relatively far apart in the reduced-dimension space.

Let's take one last trip into the realm of mathematics, with our dataset again and again $X = (x_1, x_2, ..., x_n) \subset \mathbb{R}^m$. This time with pairwise similarity $p_{i,j}$, which represents the conditional probability of choosing x_j as a neighbor of x_i under a Gaussian distribution centered at x_i defined by :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma_i^2)}{\Sigma_{k \neq i} \exp(-\|x_i - x_k\|^2/2\sigma_i^2)}$$

This function represents the similarity between our data points (figure 6) in high-dimensional space.

For the Low-Dimensional Similarity we place ourselves in the low-dimensional space $Y = \{y_1, y_2, \ldots, y_n\}$, where the similarities q_{ij} are defined similarly but in the lower-dimensional

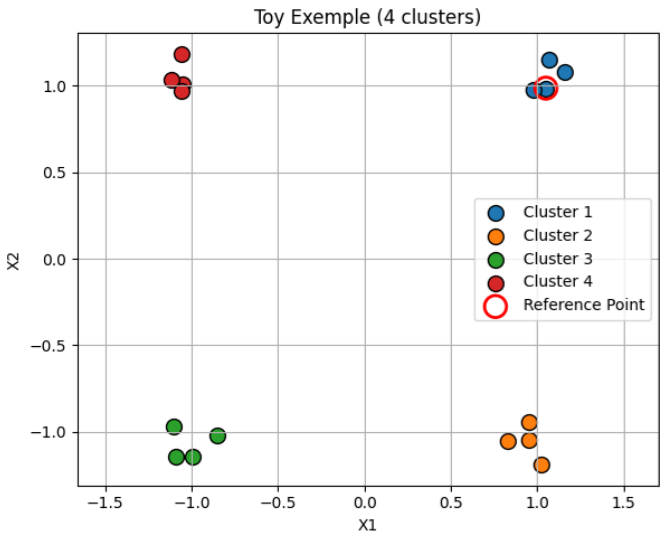

Fig. 6. Exemple of data in 2D

space using a Student t-distribution with one degree of freedom:

$$q_{j|i} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i}(1 + \|y_i - y_k\|^2)^{-1}}$$

This function is used for reduced-dimensional space, as the tail of the Gaussian function tends too quickly towards 0, which would place all distant points in the same position. We can see in figure7 a thicker tail for the Student t-distribution, which will enable us to retain information on distant points in this reduced space.

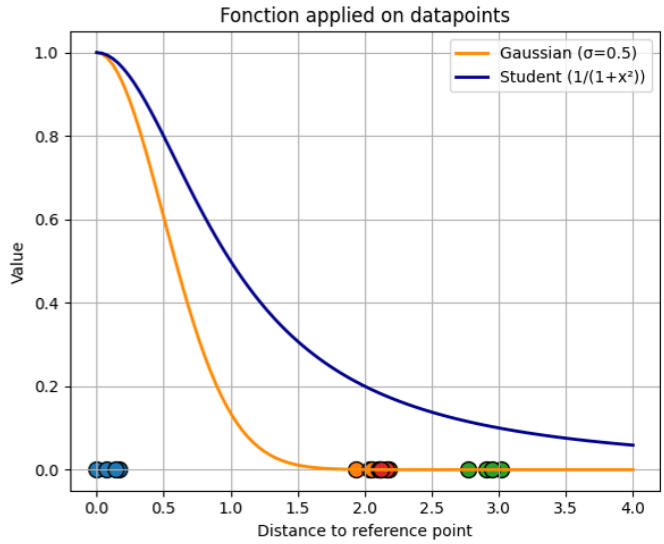

Fig. 7. Student t-Distribution compared to the Gaussian one on the toy example.

Now, the goal is to minimize the difference between the similarities of points in the high-dimensional space and their counterparts in the low-dimensional space. We measure this difference using the **Kullback-Leibler (KL) divergence**. The KL divergence measures how one probability distribution diverges from another. In our case, it quantifies how different the pairwise similarities are between high-dimensional and low-dimensional spaces.

The objective function C is defined as the Kullback-Leibler divergence between the distributions of p_{ij} and q_{ij}, weighted by a constant **Perplexity** for controlling the effective number of neighbors:

$$C = \sum_i \mathrm{KL}(P_i \| Q_i) = \sum_i \sum_j p_{i,j} \log(\frac{p_{i,j}}{q_{i,j}})$$

To minimize the KL divergence, gradient descent is employed. This iterative optimization method updates the positions of points in the low-dimensional space. At each iteration, the gradient of the cost function with respect to the positions of the points in the low-dimensional space is computed. This gradient indicates the direction in which we should move each point to reduce the difference between the high-dimensional and low-dimensional similarities. By updating the positions of points based on this gradient, the algorithm gradually converge towards a configuration where the low-dimensional similarities closely match those of the high-dimensional space.

The gradient of the objective function with respect to the low-dimensional points y_i is computed and used to iteratively update the positions of the points in the low-dimensional space.

The gradient $\frac{\partial C}{\partial y_i}$ of the cost function with respect to the low-dimensional point y_i is computed as:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

Stochastic gradient descent or its variants are typically used to optimize the objective function by updating the low-dimensional embeddings Y iteratively until convergence.

This, then, is the essence of t-SNE's mathematical operation, which aims to preserve local similarities between data points in an original high-dimensional space, while integrating them into a lower-dimensional space for visualization purposes (figure 8).

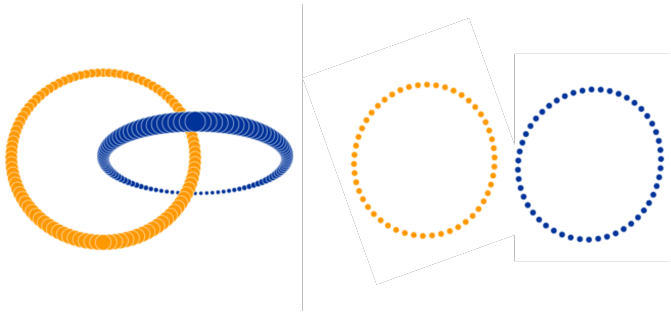

Fig. 8. Exemple of t-SNE, source : https://distill.pub/2016/misread-tsne/

note : There was not enough time to make such a plot on our own data.

4 Results

During this internship, it was very difficult to evaluate our results for the simple reason that our data meant nothing to any human being not working at STMicroelectronics. So we built our algorithms on our data without really being able

to get feedback on the relevance of our results. After a few weeks, we had a database that enabled us to link the company's components to those of the competition. The idea with this database is to be able to know whether components are in the same cluster or not. This is the only way to evaluate our clustering.

This was done with the K-Means algorithm using 13 clusters, found using the Elbow Method. We then decided to test the 3 numerical distances, keeping the Dice one for categorical features. The tests (table 2) carried out are as follows : we apply the K-Prototypes algorithm to our data. We then go through the elements of the competition and look for the clusters with the highest number of matching STMicroelectronics elements. If the cluster is the right one, we can say that the data is well clustered.

TABLE 2
Comparison of distances on the heuristic test

Distance	**Euclidean**	**Manhattan**	**Chebyshev**
Same Cluster	95165	94704	42383
Not in the Same Cluster	101225	109652	154007
Percentage	**48.4%**	48.2%	21.58%

5 Conclusion

The results obtained are a little meagre, but in just one month they are very encouraging. We were able to cover a large number of concepts that are fundamental to data processing. We have been able to study a number of metrics and their importance in the different situations imposed by the field of computer science and Machine Learning. The fact that we were able for the first time to process such a large amount of data was very impressive, and a great learning experience for the rest of our course. This first collaboration with a company was a very enriching and formative experience.

References

[1] Z. Huang, "Extensions to the k-means algorithm for clustering large data sets with categorical values," *Data Mining and Knowledge Discovery*, vol. 2, no. 3, pp. 283–304, 1998.
[2] J. Shlens, "A tutorial on principal component analysis," 2014.
[3] T. T. Cai and R. Ma, "Theoretical foundations of t-sne for visualizing high-dimensional clustered data," 2022.